WaterWatch SL: Community Water-Point Repair Prioritisation and Safe-Alternative Referral System

Module: PROG101 — Principles of Programming Logic and Design
Project type: Final project design submission
Country focus: Sierra Leone
Prepared by: Student/Group name to be inserted

1. Problem analysis

Sierra Leone has substantial water resources, but many communities still experience limited access to reliable and safe water because infrastructure and service delivery are weak 1. UNICEF's drinking-water service ladder distinguishes between basic, limited, unimproved, and surface-water services. A round trip that takes more than 30 minutes to collect water is classified as a limited service 2.

The problem is especially visible in rural communities. MSF reported in 2024 that 23% of people in Sierra Leone lacked basic water services and about 58% used unsafe sources such as rivers, dams, and unprotected wells. It also described seasonal shortages, non-functional or insufficient wells, and community training for borehole maintenance in Tonkolili 3. Sierra Leone's WASH data portal provides water-point data, maintenance guidance, and a national water-point report, showing that water-service monitoring and maintenance are recognised sector needs 4.

Problem statement

When a community water point fails, there is no simple, transparent, low-bandwidth design in this project for capturing the failure, assessing its urgency, identifying a safer nearby option, and preparing a repair referral. As a result, repair teams may receive incomplete information and communities may continue using unsafe sources.

Proposed solution

WaterWatch SL is a conceptual reporting and prioritisation system. A community reporter enters the location, water-point type, fault, number of people affected, time out of service, health/safety risk, and available alternatives. The system validates the input, calculates a priority score, classifies the case, suggests the safest available alternative, and produces a referral summary.

The design does not replace a WASH authority, laboratory testing, or emergency health advice. It organises information so that authorised people can make faster and more consistent decisions.

2. Requirements analysis

Functional requirements

1. The system shall capture a water-point report.
2. The system shall validate required fields and reject incomplete or invalid values.
3. The system shall classify the reported fault.
4. The system shall calculate a transparent priority score.
5. The system shall classify the report as Critical, High, Medium, or Low.
6. The system shall compare available alternatives by safety, distance, and availability.
7. The system shall recommend the safest available alternative.
8. The system shall generate a referral summary.
9. The system shall allow a user to restart and enter another report.
10. The system shall record a simple status: New, Referred, In Progress, Resolved, or Closed.

Non-functional requirements

- **Accessibility:** use simple wording, large controls, and support for local-language labels in later versions.
- **Low bandwidth:** the workflow must be usable with text-first forms and intermittent connectivity.
- **Transparency:** show the factors contributing to the priority score.
- **Privacy:** collect only information needed for service coordination; avoid unnecessary personal data.
- **Reliability:** preserve a report locally when connectivity is unavailable in a future implementation.
- **Maintainability:** keep scoring weights and categories in one configuration module.
- **Interoperability:** export a referral summary in a format that can be shared with authorised WASH actors.

3. Computational thinking process

Decomposition

The problem is separated into input collection, validation, risk assessment, scoring, alternative selection, referral generation, and report-status management.

Pattern recognition

Most water-point cases contain the same information: location, source type, fault, affected population, duration, safety risk, and alternatives. Similar cases can therefore use one consistent scoring method.

Abstraction

The design does not model every engineering detail of a pump. It models the minimum information needed to prioritise a service referral.

Algorithm design

The algorithm follows a repeatable sequence: collect data, validate it, score urgency, select a safe alternative, produce an action, and optionally repeat for another report.

4. Input–Processing–Output design

Stage	Design
Input	Reporter ID or role, chiefdom/community, water-point ID, source type, fault type, people affected, days unavailable, safety risk, alternatives, distance to alternatives, availability status
Processing	Validate fields; map categories to points; calculate total score; classify urgency; rank alternatives; create referral recommendation; assign status
Output	Priority score, priority class, reason breakdown, recommended alternative, response timeframe, referral summary, status

5. Scoring algorithm

The score is deliberately explainable:

`PriorityScore = PeoplePoints + RiskPoints + DowntimePoints + AlternativePoints`

The maximum is 100 points.

People affected: 0–35 points

- 0–49 people: 5 points
- 50–199 people: 15 points
- 200–499 people: 25 points
- 500 or more people: 35 points

Safety risk: 0–30 points

- No known safety concern: 0 points
- Possible contamination or minor hazard: 15 points
- Confirmed contamination, outbreak concern, or dangerous access: 30 points

Time without service: 0–20 points

- 0–1 days: 5 points
- 2–7 days: 10 points
- 8–30 days: 15 points
- More than 30 days: 20 points

Safe alternative: 0–15 points

- Safe and available alternative within 30 minutes: 0 points
- Safe alternative exists but requires more than 30 minutes: 8 points
- No confirmed safe alternative: 15 points

Priority classification

- 75–100: Critical — refer immediately
- 55–74: High — refer within 48 hours
- 30–54: Medium — schedule within 7 days
- 0–29: Low — monitor or include in routine maintenance

6. Pseudocode summary

```
START
  REPEAT
    report ← collectReport()
    IF validate(report) = FALSE THEN
      display validation errors
    ELSE
      score ← calculatePriorityScore(report)
      priority ← classifyPriority(score)
      alternative ← chooseSafeAlternative(report.alternatives)
      referral ← createReferral(report, score, priority, alternative)
      display referral
      save report with status NEW
    END IF
    ask whether another report is needed
  UNTIL answer is NO
END
```

The complete modular pseudocode is in docs/pseudocode.md.

7. Functional modules

1. **Report Intake Module** — collects report details.
2. **Validation Module** — checks required fields and valid ranges.
3. **Scoring Module** — calculates each score component and the total.
4. **Priority Module** — maps the score to a response category.
5. **Alternative Selection Module** — ranks alternatives and excludes unsafe or unavailable options.
6. **Referral Module** — creates an actionable summary for a WASH focal person.
7. **Status Module** — manages report lifecycle.
8. **User Interface Module** — presents prompts, results, and error messages.

8. Expected outputs

For a valid report, the system should produce a clear result such as:

> **Priority: High ($68/100$).** Borehole WW-014 in Community A has been unavailable for 10 days. Approximately 230 people are affected. A possible contamination concern was reported. The nearest confirmed safe alternative is 45 minutes away. Refer to the district WASH focal person within 48 hours. Status: New.

This example demonstrates the format only; values must come from user input in an implementation.

9. Flowchart design

The flowchart is provided as a native Flowgorithm file in `diagrams/waterwatch-flowchart.fprg`. Open the `.fprg` file in Flowgorithm to view the editable flowchart. It shows the main loop, validation decision, score calculation, alternative assessment, referral output, and restart decision.

10. Open-source, Digital Public Good, and SDG alignment

Why open source

The problem affects communities and service organisations that may not be able to pay for proprietary software. An open licence allows local organisations, students, councils, and NGOs to inspect, adapt, translate, and improve the design. The MIT License is used for this classroom repository.

Digital Public Good principles

WaterWatch SL is designed to be open, reusable, privacy-aware, locally adaptable, and useful for public service delivery. It uses open documentation and an explainable algorithm. A production version would need stronger safeguards, open data standards, community governance, and security review.

SDG contribution

The project most directly supports **SDG 6: Clean Water and Sanitation**, especially target 6.1 on universal and equitable access to safe and affordable drinking water. It also supports **SDG 3: Good Health and Well-being** by helping identify unsafe water risks, and **SDG 10: Reduced Inequalities** by highlighting communities facing long collection times or no safe alternative.

11. Public benefit and expected impact

The immediate public benefit is better information for repair prioritisation. A consistent report can reduce repeated trips, improve referral quality, and make it easier to see which communities lack a safe alternative. The design may also support accountability because the scoring factors are visible rather than hidden in an opaque ranking.

Expected longer-term outcomes include faster response to high-risk failures, better maintenance records, more targeted use of spare parts, and stronger community participation in water-point management. These outcomes must be evaluated with real service data; the project does not claim that the algorithm alone will solve water access problems.

12. Limitations and safeguards

The score is a classroom design assumption, not an official Sierra Leone prioritisation policy. A real system must validate the weights with the Ministry of Water Resources, district councils, WASH service providers, community representatives, and public-health stakeholders. Reports should not expose sensitive personal information. Safety messages should direct users to authorised health and WASH guidance, especially when contamination or disease outbreaks are suspected.

13. Conclusion

WaterWatch SL translates a real Sierra Leone WASH challenge into a manageable programming-logic problem. It demonstrates decomposition, input validation, decision structures, loops, modular functions, scoring, and clear outputs. Because the design is open and explainable, it can be used as a starting point for a future student prototype or community-led service tool.

References